Responsibility-Carrying State: Auditable Sufficiency, Reopen Contracts, and Safe State Reuse

ORION-P13

21 August 2026

Abstract

A compact state is not simply "sufficient" or "insufficient." Sufficiency is relative to what the downstream system is responsible for doing. A representation adequate for prediction can omit distinctions required for intervention, verification, diagnosis or repair; confidence and provenance alone do not specify that boundary. We formalize **responsibility-relative sufficiency** and introduce a non-self-authorizing `ResponsibilityCarryingState` (RCS) contract that binds compact state to the responsibilities it supports, its independent witness, explicit omissions, context assumptions, raw-state recoverability, resource envelope and reopen conditions. The paper preserves a historical negative: an earlier frozen sufficiency-debt experiment missed its finite-sample sentinel at `0.0556640625 > 0.05`; the result is retained rather than retuned. An independently frozen successor uses exact responsibility equivalence classes and a held-out reuse benchmark. Across **12,288 protected episodes**, RCS produces **0 unsafe reuses**, **0 unnecessary reopens** and **0.9807 verified correctness**. Confidence-only makes **21.56% structurally unsafe reuses**; provenance-only and unqualified compact state make **39.62%**. RCS costs **2.875** units on average versus **5.732** for always reopening raw state and emits `CANNOT_CHECK` for every unsupported/nonrecoverable case. Two runs are byte-identical. The controlled claim is that **an explicit responsibility/recovery contract can eliminate structurally unsafe state reuse without degenerating to always-reopen behavior, whereas scalar confidence and provenance do not encode the same sufficiency boundary**.

1 Introduction

Compression, abstraction and state reuse are normally evaluated against a named objective. Problems appear when compact state is reused after the downstream responsibility changes. A summary sufficient for a factual question may omit provenance needed to authorize a scientific claim. A proof-state abstraction useful for next-tactic prediction may omit dependencies needed to diagnose or repair a failed proof. A control-state abstraction adequate for one policy may collapse distinctions needed for intervention or counterfactual analysis.

This is not merely an uncertainty problem. A system can be highly confident while state omits a variable that becomes decisive under a new responsibility. Nor is it merely provenance: knowing exactly where state came from does not establish what that state can safely support.

P13 asks:

> **Can a compact state carry a machine-checkable responsibility contract that says what it supports, what it omits, what changes invalidate reuse, and whether richer state can be reopened—without forcing every decision to reload raw evidence?**

The paper contributes responsibility-relative sufficiency; a responsibility-carrying state interface; permanent negative-result analysis; and independent controlled safety–cost superiority evidence.

2 Donor boundary and responsibility-relative sufficiency

Statistical sufficiency, state abstraction, bisimulation, predictive-state representations and causal abstractions already establish that different tasks can require different retained distinctions. Selective prediction and uncertainty gating use confidence to abstain. Provenance/evidence tracing binds artifacts to origin. Proof-carrying systems attach verifiable certificates. Memory-staleness work asks when stored information is no longer valid. P13 claims none of these primitives.

The residual is **responsibility-scoped certified reuse with explicit reopen semantics and a measured interior safety–cost advantage**.

Let raw world `X` induce correct output `g_rho(X)` for responsibility `rho`, and let compact representation be `Z=T(X)`.

2.1 Exact responsibility sufficiency

`Z` is sufficient for responsibility `rho` over world set `Omega` if there exists `h_rho` such that `g_rho(x)=h_rho(T(x))` for every `x` in `Omega`. Equivalently, every equivalence class induced by `T` must be homogeneous under `g_rho`.

2.2 Responsibility-shift witness

A pair `(x,x')` witnesses insufficiency after `rho_L -> rho_H` when `T(x)=T(x')`, the lower-responsibility outputs agree, and the higher-responsibility outputs differ. No learner is needed to establish such a witness; it is a property of the abstraction and responsibility.

For empirical systems, operational sufficiency debt is the verified higher-responsibility performance gap between richer and compact state, conditional on a prospectively frozen lower-responsibility equivalence/noninferiority requirement. It is a benchmark quantity, not a universal information measure.

3 Permanent negative history

The historical experiment constructed independent binary latent variables `(x,m,r)` and responsibilities `PREDICT=x`, `DECIDE=x`, `INTERVENE=x*m`, `VERIFY=x*m`, and `REPAIR=r`. Representations were `Z1=(x)`, `Z2=(x,m)` and `Z3=(x,m,r)`.

Exact enumeration produced the intended responsibility ladder: all representations were perfect for prediction/decision; `Z1` was exactly 0.5 on intervene/verify while `Z2/Z3` were 1.0; `Z2` was 0.5 on repair while `Z3` was 1.0. Exact upward debts were therefore +0.50.

However, the frozen protocol also required the maximum deviation among 100 finite-sample sanity replicates of `n=1024` to be ≤ 0.05. The observed maximum was `0.0556640625` at replicate 92. The combined terminal remains permanently `P14_CONTROLLED_SUFFICIENCY_DEBT_GATE_NOT_MET`.

3.1 Root cause

The harness grouped a finite sample by compact state, selected the majority target value from that same sample and credited that majority on the same observations, then maximized deviation

over 100 replicates. The statistic combines within-sample majority optimism with an extreme-value operation. The correct response is not to change `0.05` after observing `0.0556640625`. The old terminal stays negative. The successor changes the estimand: exact support is evaluated by exhaustive equivalence classes, while efficacy is evaluated on a fresh safety–cost benchmark.

4 ResponsibilityCarryingState contract

An RCS contains compact state plus a fail-closed contract binding raw/source evidence identity; compiler/transform identity and version; exact supported responsibility set; independent witness/certificate identity; intentionally omitted coordinates/information classes; required-same context coordinates; reopen-on-change coordinates; raw recovery/reconstruction availability and freshness; recovery/reopen cost; resource envelope; evaluator identity; authority owner; and an explicit declaration that the object grants no scientific/novelty self-authority.

At reuse time the contract returns:

- `USE_COMPILED` when requested responsibility is supported and all bound conditions hold;
- `REOPEN_REQUIRED` when support does not hold but richer state can be recovered;
- `CANNOT_CHECK` when support is absent and recovery/verification cannot establish a safe route.

`CANNOT_CHECK` is a correct scientific disposition, not an error to hide.

5 Independent protected benchmark

The successor retains the interpretable world but evaluates support exactly: `Z1` supports {`PREDICT, DECIDE`}; `Z2` supports {`PREDICT, DECIDE, INTERVENE, VERIFY`}; and `Z3` supports all five responsibilities. The exact equivalence-class matrix is checked directly and matches the registered support sets.

Protected seed: `2026082113`. Families: **24**. Episodes per family: **512**. Total protected episodes: **12,288**.

Family probabilities for hidden variables `m` and `r` vary independently over `[0.65,0.95]`. Compact state is `Z1` or `Z2` with equal probability; requested responsibility is uniform over five tasks. Raw recovery is independently available with probability `0.95`. When compact state lacks a needed coordinate, an unqualified decoder uses the family MAP value, deliberately creating cases that can be high-confidence yet structurally unsupported.

5.1 Baselines

1. `UNQUALIFIED`: always reuse compact state.
2. `CONFIDENCE_ONLY`: reuse if estimated task accuracy is at least `0.80`; otherwise reopen when possible.
3. `PROVENANCE_ONLY`: valid lineage is present, so provenance alone permits reuse.
4. `RCS`: reuse only when exact responsibility support holds; otherwise reopen if raw is recoverable, else `CANNOT_CHECK`.
5. `ALWAYS_RAW`: reopen raw state whenever possible.

Fixed resource units are `REUSE=1`, `REOPEN=6`, `CANNOT_CHECK=0.5`.

Baselines 1 and 3 are the same policy on this corpus, and the list is four distinct policies rather than five. Every episode supplies valid lineage by construction, so the provenance check never

refuses and PROVENANCE_ONLY reduces to "always reuse" — which is UNQUALIFIED. Their measured rates are identical on every metric reported: unsafe reuse 0.3961588541666667, verified correctness 0.9248046875, mean cost 1.0. This follows from the construction above rather than from anything that happened during the run, and the results report the two together for that reason. It has one consequence worth stating plainly: this benchmark does not test whether a provenance check helps, because the check is never exercised. Establishing that would need episodes with absent or broken lineage, which this corpus does not contain.

6 Results

Protected terminal: P13A_RCS_SAFETY_COST_SUPERIORITY_SUPPORTED.

arm	unsafe reuse	verified correctness	unnecessary reopen	mean cost
RCS	**0.0000**	**0.9807**	**0.0000**	**2.8747**
confidence only	0.2156	0.9657	lower than RCS	1.8582
provenance only	0.3962	0.9248	0	1.0000
unqualified compact	0.3962	0.9248	0	1.0000
always raw	0.0000	0.9513	0.5744	5.7319

RCS emits CANNOT_CHECK for all **237** unsupported/nonrecoverable cases and no other protected case. It eliminates structural unsafe reuse without adopting always-reopen behavior. Mean resource cost is approximately **49.8% lower** than always raw. Two fresh executions are byte-identical with SHA-256 ea4006981e0c5027a56789014dd723059420f603e071e81990a903986f6e8d1f.

6.1 Why confidence fails

The omitted coordinate is biased within a family, so a MAP decoder can be highly accurate and exceed the 0.80 confidence threshold. But high expected accuracy does not make an unsupported equivalence class sufficient. Confidence asks how often an output is right under a distribution; RCS asks whether state retains distinctions required by a responsibility.

6.2 Why provenance fails

Every compact state has valid lineage. Provenance verifies origin but says nothing about whether m or r was retained. Provenance-only reuse is structurally unsafe on 39.62% of protected episodes.

6.3 Why always raw is not the answer

Always reopening prevents unsafe compact reuse but pays roughly twice the mean RCS cost and unnecessarily reopens supported cases on 57.44% of episodes. RCS occupies the desired interior safety–cost point.

7 Certificate transport, related work and reproducibility

Responsibility support is conditional on evidence identity, transform version, required context, witness identity and resource envelope. A semantic change can require preserve, reopen, revoke or CANNOT_CHECK behavior. The RCS object does not certify its own scientific authority; an evaluator can establish operational support without granting novelty, publication or safety-critical deployment authority.

Recent proof-carrying agent-action work attaches model-agnostic certificates to actions and runtime governance. Provenance traces evidence and execution. Memory-staleness systems detect that stored state is no longer valid. These donors make it insufficient to claim simply that "state should carry a certificate." P13's discriminator is the **responsibility key plus reopen semantics**: a state may be current, well-provenanced and high-confidence yet insufficient for another downstream responsibility.

The support matrix is deterministic and exact. The historical negative is reported exactly as frozen and is not reanalysed into a positive result. P13A uses fresh seed/families and a different primary estimand. The paper reports protected rates and deterministic replay rather than adding post-hoc inferential tests. A real-system extension should define task/domain as the unit of generalization and use paired matched comparisons with family/domain-block uncertainty.

8 Limitations and conclusion

Responsibilities are discrete and known in the controlled world; real systems may have ambiguous or compositional responsibilities. Exact sufficiency is strong; approximate support needs frozen tolerances and calibrated witnesses. The contract is only as reliable as external witness and recovery metadata. Resource costs are controlled units, not measured real latency/tokens/IO. High-risk domains may rationally choose always-raw. The paper has not yet demonstrated verifier-backed Lean repair/diagnosis or blinded scientific-workflow responsibility shifts, and certificate transport/revocation under real semantic version changes remains an external validation gate.

Sufficiency is a contract over future responsibility, not an intrinsic property of compact representation. P13 makes that boundary explicit, preserves a preregistered historical failure, replaces its indirect sentinel with exact equivalence-class semantics, and shows in an independent protected benchmark that responsibility-carrying state can eliminate unsafe compact reuse without paying the cost of always reopening raw evidence. Confidence and provenance remain useful signals, but neither specifies what distinctions state supports.

A compact state becomes safely reusable only when its authority is scoped to a named responsibility and its reopen conditions are explicit.

9 References

- Classical statistical sufficiency, state-abstraction, bisimulation, predictive-state and causal-abstraction literatures are donor-owned foundations.
- Wang, Z. *Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems.* arXiv:2606.04104, 2026.
- Chao, H., Bai, Y., Sheng, R., Li, T. & Sun, Y. *STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?* arXiv:2605.06527, 2026.
- Wang, Y. et al. *From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents.* arXiv:2606.04990, 2026.
- Doyle, J. *A Truth Maintenance System.* Artificial Intelligence 12(3):231–272, 1979.
- de Kleer, J. *An Assumption-Based TMS.* Artificial Intelligence 28(2), 1986.